

Your first JSON views

Write Django views that read what the client sent and answer with JSON and the right status code.

The big idea

In exercise 1 you split a raw request by hand. Django does that for you, then **calls a function you wrote** and passes the request in. That function is a **view**.

A view is like a clerk at a counter: take the form, check it, hand back a reply slip with a stamp on top: "OK", "filled in wrong" or "wrong counter". The stamp is the **status code**.

What the tests send

You don't build `request` yourself: the tests make a fake one and call your view with it.

```
factory = RequestFactory()

greet(factory.get("/greet/", {"name": "Ada"}))    # like GET /greet/?name=Ada

add(factory.post("/add/", data='{"a": 2, "b": 3.5}',
                content_type="application/json")) # a POST with a JSON body
```

What your views must return

The tests check the status code and the JSON. Error messages must match exactly.

```
GET    /greet/?name=Ada           200    {"message": "Hello, Ada!"}
GET    /greet/?name= Grace       200    {"message": "Hello, Grace!"}
GET    /greet/    (no name)      400    {"error": "name is required"}
POST   /greet/                               405    (Django fills this in)
POST   /add/    {"a": 2, "b": 3.5}        200    {"result": 5.5}
POST   /add/    {not json                400    {"error": "invalid JSON"}
POST   /add/    {"a": "1", "b": 2}        400    {"error": "a and b must be numbers"}
GET    /add/                               405    (Django fills this in)
```

New words

view

A function that receives a request and returns a response.

request object

The request, already split up: `request.method`, `request.GET`, ...

JSON

Text data that looks like a dict: `{"message": "Hello"}`.

status code

`200` OK, `400` client sent something wrong, `405` wrong method.

query / body

Values in the address after `?` / data sent inside the request.

decorator

An `@` line just above `def` that adds behaviour.

Tool: request.GET.get

`request.GET` holds the query. `.get(name, default)` gives the default instead of crashing.

```
request = RequestFactory().get("/weather/", {"city": " Paris "})
print(request.method)
print(request.GET.get("city", ""))
print(request.GET.get("country", "unknown"))
print(request.GET.get("city", "").strip())
```

OUTPUT

GET

Paris

unknown

Paris

Tool: JsonResponse

A class you call like a function. The status is 200 unless you pass `status=`.

```
response = JsonResponse({"error": "city is required"}, status=400)
print(response.status_code)
print(response.content)
print(JsonResponse({"temp": 21}).status_code)
```

OUTPUT

```
400
b'{"error": "city is required"}'
200
```

Tool: @require_POST and @require_GET

Write it on the line directly above `def`. A wrong method gets a 405 and your code never runs.

```
@require_POST
def ping(request):
    return JsonResponse({"pong": True})

print(ping(RequestFactory().post("/ping/")).status_code)
print(ping(RequestFactory().get("/ping/")).status_code)
```

OUTPUT

200

405

Tools: json.loads and try / except

```
body = b'{"item": "tea", "qty": 2}'      # what request.body looks like
data = json.loads(body)
print(data["qty"] * 10)
print(data.get("price"))

try:
    json.loads("{oops}")
except json.JSONDecodeError:
    print("That was not valid JSON")
```

OUTPUT

```
20
None
That was not valid JSON
```

Query vs body

request.GET

- Values in the **address**, after `?`
- Already split up for you
- Always **text**
- Read with `.get("name", "")`
- Used by `greet`

request.body

- Data sent **inside** the request
- Raw **bytes**
- Parse it with `json.loads(...)`
- Numbers stay numbers
- Used by `add`

Pattern: check, then use

The same shape you need in `add`, on different data (just the logic, not a view):

```
data = {"width": 4, "height": "tall"}
w, h = data.get("width"), data.get("height")
if not isinstance(w, (int, float)) or not isinstance(h, (int, float)):
    print("width and height must be numbers")
else:
    print(w * h)
```

OUTPUT

```
width and height must be numbers
```

Your task, step by step

- 1 Put `@require_GET` directly above `def greet` → **test 4**
- 2 Read the name with `request.GET.get("name", "").strip()`
- 3 If the name is empty, return the error with `status=400` → **test 3**
- 4 Otherwise return `{"message": f"Hello, {name}!"}` → **tests 1-2**
- 5 Put `@require_POST` directly above `def add` → **test 8**
- 6 `json.loads(request.body)` inside `try`; bad JSON gives a 400 → **test 6**
- 7 If `a` or `b` is not an int or float, return a 400 → **test 7**
- 8 Return `{"result": ...}` with the sum → **test 5**

Common mistakes

- Error dict without `status=400`: the JSON looks right, the status is 200
- `request.POST` for JSON: it only holds form data; use `request.body`
- `if a and b:` isn't a number check: `"1"` is true, `0` is false
- `and` instead of `or`: the error is needed if **either** value is bad
- A typo like `"Name is required"`: the tests compare text exactly

Where you'll use this

A shop's mobile app sends `POST /api/cart/` with `{"item": 42, "qty": "two"}`. Your view must spot the bad quantity and reply **400** with a clear message, not crash with a **500**.

- Every Django API endpoint is a view: read, validate, return JSON and a status
- The app shows your error message to the user, so it must be clear
- Django REST Framework later does the parsing and checking for you, in the same pattern

Check yourself

1. What does `request.GET.get("name", "")` give when the name is missing?
2. Which status fits a missing name: 200, 400, 405 or 500? Why?
3. Where is a JSON body in a plain Django view, and how do you read it?
4. Why is `isinstance(value, (int, float))` safer than `if value:`?

Keep learning

[Django views – time to create!](#)

Django Girls Tutorial · tutorial · beginner. Start here: a gentle first look at what a view is (a function that takes a request and returns a response).

[HTTP response status codes](#)

MDN Web Docs · docs · beginner. Look up what 200, 400 and 405 mean so you pick the right status for each response.

[json: JSON encoder and decoder](#)

Python docs · docs · beginner. The reference for `json.loads`, which turns the request body into a Python dict, and the error it raises on bad input.

[Request and response objects \(JsonResponse\)](#)

Django docs · docs · intermediate. The official reference for `request.GET`, `request.body` and `JsonResponse`, the exact tools your views use.

[View decorators: allowed HTTP methods](#)

Django docs · docs · intermediate. Explains `require_GET` and `require_POST`, which reject the wrong method with a 405 for you.